DANESHVAR AMROLLAHI

daneshvar@cs.stanford.edu cs.stanford.edu/~daneshva github.com/daneshvar-amrollahi
389 Jane Stanford Way, CoDa #W310, Stanford, CA 94305, United States

EDUCATION

- **Stanford University** *2024/01 - Present*
  PhD in Computer Science (Advisor: Clark Barrett)
- **Stanford University** *2024/01 - 2025/06*
  MSc in Computer Science
- **University of Tehran** *2018/09 - 2023/02*
  BSc in Computer Engineering (Software) *GPA: 18.02/20.00*

PUBLICATIONS

- D. Amrollahi, M. Preiner, A. Niemetz, A. Reynlods, M. Charikar, C. Tinelli, C. Barrett (2024). **Towards SMT Solver Stability via Input Normalization.** *Formal Methods in Computer-Aided Design (FMCAD 2025).*
- P. Hozzová, D. Amrollahi, M. Hajdu, L. Kovács, A. Voronkov, E.M. Wagner (2024). **Synthesis of Recursive Programs in Saturation.** *International Joint Conference on Automated Reasoning (IJCAR 2024).*
- D. Amrollahi, E. Bartocci, G. Kenison, L. Kovács, M. Moosbrugger, M. Stankovič (2024). **(Un)Solvable Loop Analysis**. *Formal Methods in System Design (FMSD).*
- D. Amrollahi, H. Hojjat, P. Rümmer (2024). **An Encoding for CLP Problems in SMT-LIB**. *10th Workshop on Horn Clauses for Verification and Synthesis (HCVS 2023).*
- D. Amrollahi, E. Bartocci, G. Kenison, L. Kovács, M. Moosbrugger, M. Stankovič (2022). **Solving Invariant Generation for Unsolvable Loops**. *29th International Static Analysis Symposium (SAS 2022).* **Awarded the Radhia Cousot Young Researcher Best Paper Award.**
- A. Humenberger, D. Amrollahi, N. Bjørner, L. Kovács (2022). **Algebra-Based Reasoning for Loop Synthesis**. *Formal Aspects of Computing (FAC).*

INDUSTRY EXPERIENCE

- **Applied Science Intern at Amazon Web Services (AWS)** *Santa Clara, United States*
  Under Dr. Bruno Dutertre *2025/06 - 2025/09*
  Working part of the Automated Reasoning Group on securing Amazon's services using the Lean theorem prover.

RESEARCH EXPERIENCE

- **PhD student at Center for Automated Reasoning, Stanford University** *Stanford, United States*
  Under Prof. Clark Barrett *2024/01 - Present*
  Designed and implemented a normalization algorithm for SMT-LIB queries as a pre-processing step for SMT solvers. Despite the graph-isomorphism hardness of the problem, the algorithm consistently produces similar outputs across semantically identical mutations of a query. Implemented in cvc5, it has improved the stability of both cvc5 and Z3 on diverse benchmarks, including large-scale real-world industrial (Dafny and F*) program verification tasks.
- **Research Intern at Dependable Systems Lab, EPFL** *Lausanne, Switzerland*
  Under Prof. George Candea *2022/07 - 2022/08*
  Enhanced PIX, a tool for extracting performance interfaces for network functions (NFs), by extending KLEE's symbolic execution engine with Z3's first-order logic quantifiers. Implemented loop summaries to reduce path explosion caused by loops, significantly improving the scalability of symbolic execution on libc string functions commonly used in network software processing packets. This work required strong programming skills in C++ for modifying KLEE's codebase and integrating it with Z3.

- **Research Intern at Automated Program Reasoning Group, TU Wien** *Vienna, Austria*
  Under Prof. Laura Kovács and Prof. Ezio Bartocci *2021/07 - 2022/02 + 2023/05 - 2023/12*

  1. Extended state-of-the-art techniques for generating polynomial loop invariants to support a broader class of loops. Implemented these advancements in Polar, a static analyzer designed for reasoning about probabilistic while-loops, using techniques from symbolic computation. The work won the **Radhia Cousot Young Research Best Paper Award** at the 29th Static Analysis Symposium (SAS).
  2. Implemented a recursive program synthesis framework based on mathematical induction over algebraic datatypes, into Vampire, a state-of-the-art theorem prover written in C++ used for verification applications.

- **Research Intern at Programming Methodology Group, ETH Zürich** *Zürich, Switzerland*
  Under Prof. Peter Müller *2022/03 - 2022/04*
  Worked on developing a concurrency-focused methodology for verifying Golang programs with global variables and package initialization, using separation logic.

HONORS AND AWARDS

- **Radhia Cousot Young Researcher Best Paper Award** *2022*
  29th Static Analysis Symposium (SAS 2022)
- **Stanford School of Engineering (SoE) Fellowship** *2024*
  Awarded a $12900/quarter stipend and full tuition coverage for one year
- **Ranked 8th** in **ACM-ICPC (International Collegiate Programming Contest)** *2020*
  West Asia Region, Tehran site
- **Summer@EPFL Fellowship** *2022*
  Ranked top 1.5% among 4000 applicants and awarded a 1600 CHF/month fellowship over summer

PROJECTS

- **cvc5** — github.com/daneshvar-amrollahi/cvc5/tree/normalize *C++*
  Implemented a query normalizer, improving performance stability. This is an industrial-strength SMT solver.
- **Vampire** — github.com/vprover/vampire/tree/synthesis-recursive *C++*
  Implemented a framework within a saturation-based first-order theorem prover for synthesizing recursive programs using structural induction over algebraic datatypes.
- **Polar** — github.com/probing-lab/polar *Python, SymPy*
  Implemented a polynomial loop-invariant synthesizer for (probabilistic) unsolvable loops, using recurrences.
- **KLEE** — github.com/bolt-perf-contracts/klee/pull/9 *C++, Z3*
  Integrated Z3's support for existential/universal quantifiers into the KLEE symbolic execution engine codebase to summarize loops using quantified formulas in first-order logic, and mitigate the path explosion problem.
- **Koloocheh** — github.com/daneshvar-amrollahi/Koloocheh *Pyfthon, gRPC*
  A peer-to-peer file-sharing system employs the flooding algorithm for search operations and ensures a small graph diameter by leveraging random graph properties.
- **xv6** — github.com/daneshvar-amrollahi/os-lab-xv6 *C*
  Extended xv6 operating system kernel with several extra features such as various new system calls, multilevel queue scheduling, and synchronization.
- **FTP Server** — github.com/daneshvar-amrollahi/FTP-Server *C++, Posix Threads*
  A multithreaded client-server implementation of an FTP Server using sockets for communication.

SKILLS

- **Programming Languages**:
  - Experienced in C, C++, Python.
  - Familiar with Scala, Rust, Go, Bash, SystemVerilog.

- **Tools**: cvc5, Z3, Lean, KLEE, LaTeX.